



Σχεδιασμός Ενσωματωμένων Συστημάτων

9ο εξάμηνο, Ακαδημαϊκή περίοδος 2025-2026

1η Εργαστηριακή Άσκηση

Αγγελική Ζέρβα: 03121101

Δημήτρης Καμπανάκης: 03121012

Περιεχόμενα

Εισαγωγή	3
Ζητούμενο 1ο	3
1.1	3
1.2	3
1.3	4
1.4	12
1.5	13
Ζητούμενο 2ο	16
2.1	16
2.2	16
2.3	16
2.4	19

2.5	22
---------------	----

Εισαγωγή

Στόχος της συγκεκριμένης εργαστηριακής άσκησης μελετάται είναι η βελτιστοποίηση αλγορίθμων και εφαρμογών με στόχο τη μείωση της ενεργειακής κατανάλωσης και τη βελτίωση της απόδοσης σε ενσωματωμένα και προσομοιωμένα συστήματα. Στο πρώτο μέρος της άσκησης πραγματοποιείται ανάλυση και βελτιστοποίηση του αλγορίθμου εκτίμησης κίνησης PHODS, μετρώντας και συγκρίνοντας τους χρόνους εκτέλεσης πριν και μετά από διάφορους μετασχηματισμούς κώδικα. Στο δεύτερο μέρος, μέσω του προσομοιωτή Gem5, εξερευνώνται διαφορετικές αρχιτεκτονικές ρυθμίσεις και τεχνικές βελτιστοποίησης (όπως loop unrolling και cache sizing) για την εφαρμογή tables.c, με στόχο τον εντοπισμό βέλτιστων συνδυασμών παραμέτρων.

Ζητούμενο 1ο

1.1

Μέσω του datasheet για το Renesas S7G2 και του e2studio βρήκαμε τα εξής χαρακτηριστικά του board:

Table 1: Χαρακτηριστικά του μικροελεγκτή Renesas S7G2

Παράμετρος	Τιμή / Περιγραφή
Μνήμη RAM	640 KB SRAM
Standby RAM	8 KB
Code Flash Memory	4 MB
Data Flash Memory	64 KB
FCACHE1	256 bytes (για CPU instruction fetches)
FCACHE2	16 bytes (για CPU operand access και DMAC/DTC/EDMAC/graphic IP)
FLPF (Prefetch Cache)	32 bytes (για προφόρτωση εντολών CPU)
Συχνότητα Ρολογιού	240 MHz

1.2

Στο στάδιο αυτό της άσκησης πραγματοποιήσαμε μετασχηματισμός του αρχικού κώδικα με στόχο τη μέτρηση του χρόνου εκτέλεσης της συνάρτησης `phods_motion_estimation`. Η μέτρηση υλοποιήθηκε μέσω του μετρητή κύκλων ρολογιού (DWT-CYCCNT), ο οποίος ενεργοποιήθηκε και αρχικοποιήθηκε κατάλληλα. Ο συνολικός αριθμός κύκλων διαιρέθηκε με τη συχνότητα λειτουργίας του μικροελεγκτή ώστε να προκύψει ο χρόνος εκτέλεσης σε δευτερόλεπτα. Επίσης, προσθέσαμε στον κώδικα ένα loop ώστε η συνάρτηση να εκτελείται 10 φορές και να υπολογίζεται ο μέσος, ο μέγιστος και ο ελάχιστος χρόνος εκτέλεσης για την αξιολόγηση της απόδοσης του αλγορίθμου. Παρακάτω παρουσιάζεται το κομμάτι του κώδικα που αλλάξαμε:

```
void hal_entry(void) {
// Code to initialize the DWT->CYCCNT register
CoreDebug->DEMCR |= 0x01000000;
ITM->LAR = 0xC5ACCE55;
DWT->CYCCNT = 0;
DWT->CTRL |= 1;
/* Initialize your variables here */

int motion_vectors_x[N/B][M/B], motion_vectors_y[N/B][M/B], i, j, cycles, start,
end, tot_cycles, max_cyc, min_cyc;
```

```

float mean_time, max_time, min_time;

min_cyc = MAX_INT;
max_cyc = 0;
tot_cycles = 0;

for(uint8_t it=0; it<10; ++it){
    /* Add timer code here */
    start = DWT->CYCCNT;
    phods_motion_estimation(current,previous,motion_vectors_x,motion_vectors_y);
    end = DWT->CYCCNT;

    cycles = end - start;

    if (max_cyc <= cycles)
    {
        max_cyc = cycles;
    }

    if(min_cyc >= cycles)
    {
        min_cyc = cycles;
    }

    tot_cycles += cycles;
}

max_time = (float) max_cyc / Fclk;
min_time = (float) min_cyc / Fclk;
mean_time = (float) tot_cycles / Fclk / 10.0;

// while(1){

// }
}

```

Μετά την εκτέλεση του κώδικα οι μετρήσεις που καταγράψαμε ήταν οι εξής:

Table 2: Χρόνοι εκτέλεσης της συνάρτησης `phods_motion_estimation`

Μετρική	Χρόνος (s)
Μέσος χρόνος εκτέλεσης	0.00184962545
Μέγιστος χρόνος εκτέλεσης	0.00184962922
Ελάχιστος χρόνος εκτέλεσης	0.00184962503

1.3

Με δεδομένο τον κώδικα που υλοποιήσαμε στο προηγούμενο ερωτημα για τη μέτρηση του χρόνου, θα εφαρμόσουμε διάφορους μετασχηματισμούς στον κώδικα ώστε να μειωθεί ο χρόνος εκτέλεσης. Για κάθε μετασχηματισμό θα επαναλάβουμε τη διαδικασία που περιγράψαμε παραπάνω και θα παρουσιάσουμε τους αντίστοιχους χρόνους εκτέλεσης.

Αρχικά, παρατήρουμε ότι το αρχικό διπλό loop που αρχικοποιεί τα vectors μπορεί να γίνει merge με το επόμενο διπλό loop που διατρέχει τα x,y. Επίσης, για τα δύο loop που διατρέχουν τα k και l μπορούμε επίσης να εφαρμόσουμε την τεχνική merge, καθώς δεν υπάρχουν εξαρτήσεις μεταξύ τους. Έτσι, η τελική μορφή του κώδικα με την εφαρμογή των δύο loop merge είναι η εξής:

```
void phods_motion_estimation(const int current[N][M], const int previous[N][M],
    int vectors_x[N/B][M/B],int vectors_y[N/B][M/B])
{
    int x, y, i, j, k, l, p1, p2, q2, distx, disty, S, min1, min2, bestx, besty;

    distx = 0;
    disty = 0;

    /*For all blocks in the current frame*/
    for(x=0; x<N/B; x++)
    {
        for(y=0; y<M/B; y++)
        {
            S = 4;
            //Loop Mering (Deleted upper Loop)
            vectors_x[x][y] = 0;
            vectors_y[x][y] = 0;

            while(S > 0)
            {
                min1 = 255*B*B;
                min2 = 255*B*B;

                /*For all candidate blocks in X and Y dimension*/
                //Loop Merging for the X and Y search loops
                for(i=-S; i<S+1; i+=S)
                {
                    distx = 0;
                    disty = 0;

                    /*For all pixels in the block*/
                    for(k=0; k<B; k++)
                    {
                        for(l=0; l<B; l++)
                        {
                            p1 = current[B*x+k][B*y+l];

                            if((B*x + vectors_x[x][y] + i + k) < 0 ||
                                (B*x + vectors_x[x][y] + i + k) > (N-1) ||
                                (B*y + vectors_y[x][y] + l) < 0 ||
                                (B*y + vectors_y[x][y] + l) > (M-1))
                            {
                                p2 = 0;
                            } else {
                                p2 = previous[B*x+vectors_x[x][y]+i+k][B*y+vectors_y[x][y]+l];
                            }

                            if((B*x + vectors_x[x][y] + k) < 0 ||
                                (B*x + vectors_x[x][y] + k) > (N-1) ||
```


Table 3: Χρόνοι εκτέλεσης της με βελτιστοποίηση loop merge

Μετρική	Χρόνος (s)
Μέσος χρόνος εκτέλεσης	0.00163962208
Μέγιστος χρόνος εκτέλεσης	0.00163963332
Ελάχιστος χρόνος εκτέλεσης	0.00163962087

Στη συνέχεια παρατηρούμε ότι πολλές πράξεις επαναλαμβάνονται στο εσωτερικό των loops με αποτέλεσμα να χρειάζονται περισσότεροι κύκλοι για την ολοκλήρωση του προγράμματος. Έτσι, εφαρμόζουμε μια τεχνική **data reuse**, υλοποιώντας ένα μεγάλο αριθμό των πράξεων στο εξωτερικό των loops και επαναχρησιμοποιώντας τα αποτελέσματά τους, τα οποία αποθηκεύουμε σε μεταβλητές. Επίσης, αξίζει να σημειώσουμε ότι πέρα από το data reuse, μετακινήσαμε κάποιες εντολές (όταν ήταν εφικτό) σε υψηλότερα επίπεδα, ώστε να αποφύγουμε περιττές επαναλήψεις τους.

```
void phods_motion_estimation(const int current[N][M], const int previous[N][M],
    int vectors_x[N/B][M/B], int vectors_y[N/B][M/B])
{
    int x, y, i, j, k, l, p1, p2, q2, distx, disty, S, min1, min2, bestx, besty;

    distx = 0;
    disty = 0;

    //Data Reuse - Define frequently used data outside loops
    int mult_Bx, mult_By, Bx_vector, By_vector, Bx_vector_i, By_vector_i, mult_Bx_k;
    int upper_x = N/B;
    int upper_y = M/B;
    int min_init = 255*B*B;

    /*For all blocks in the current frame*/
    for(x=0; x<upper_x; x++)
    {
        for(y=0; y<upper_y; y++)
        {
            //Loop Mering (Deleted upper Loop)
            vectors_x[x][y] = 0;
            vectors_y[x][y] = 0;
            S = 4;

            mult_Bx = B*x;
            mult_By = B*y;

            while(S > 0)
            {
                min1 = min_init;
                min2 = min_init;

                /*For all candidate blocks in X and Y dimension*/
                //Loop Merging for the X and Y search loops
                for(i=-S; i<S+1; i+=S)
                {
                    distx = 0;
                    disty = 0;

                    /*For all pixels in the block*/
```

```

for(k=0; k<B; k++)
{
    Bx_vector = mult_Bx + vectors_x[x][y]+k;
    By_vector = mult_By + vectors_y[x][y];
    Bx_vector_i = mult_Bx + vectors_x[x][y] +k + i;
    By_vector_i = mult_By + vectors_y[x][y] + i;
    mult_Bx_k = mult_Bx + k;

    for(l=0; l<B; l++)
    {
        p1 = current[mult_Bx_k][mult_By+1];

        if((Bx_vector_i) < 0 ||
            (Bx_vector_i) > (N-1) ||
            (By_vector + 1) < 0 ||
            (By_vector + 1) > (M-1))
        {
            p2 = 0;
        } else {
            p2 = previous[Bx_vector_i][By_vector + 1];
        }

        if((Bx_vector) <0 ||
            (Bx_vector) > (N-1) ||
            (By_vector_i + 1) < 0 ||
            (By_vector_i + 1) > (M-1))
        {
            q2 = 0;
        } else {
            q2 = previous[Bx_vector][By_vector_i + 1];
        }

        if(p1-p2>0){
            distx += p1-p2;
        }
        else{
            distx += p2-p1;
        }
        if(p1-q2>0){
            disty += p1-q2;
        }
        else{
            disty += q2-p1;
        }
    }
}

if(distx < min1)
{
    min1 = distx;
    bestx = i;
}

```



```

\
for(l=0; l<B; l++)\
{
    p1 = current[mult_Bx_k][mult_By+1];\
    if((Bx_vector_i) < 0 ||\
        (Bx_vector_i) > (N-1) ||\
        (By_vector + 1) < 0 ||\
        (By_vector + 1) > (M-1))\
    {
        p2 = 0;\
    } else {\
        p2 = previous[Bx_vector_i][By_vector + 1];\
    }\
    if((Bx_vector) < 0 ||\
        (Bx_vector) > (N-1) ||\
        (By_vector_i + 1) < 0 ||\
        (By_vector_i + 1) > (M-1))\
    {
        q2 = 0;\
    } else {\
        q2 = previous[Bx_vector][By_vector_i + 1];\
    }\
    if(p1-p2>0){\
        distx += p1-p2;\
    }\
    else{\
        distx += p2-p1;\
    }\
    if(p1-q2>0){\
        disty += p1-q2;\
    }\
    else{\
        disty += q2-p1;\
    }\
}\
}\
if(distx < min1)\
{\
    min1 = distx;\
    bestx = i;\
}\
if(disty < min2)\
{\
    min2 = disty;\
    besty = i;\
}\
}

#define OUTER_LOOP(S)\
    vectors_x[x][y] = 0;\
    vectors_y[x][y] = 0;\
    INNER_LOOP(-S)\
    INNER_LOOP(0)\
    INNER_LOOP(S)\

```

```
vectors_x[x][y] += bestx;\
vectors_y[x][y] += besty;\
```

Έχοντας ορίσει αυτά τα macros, το κύριο μέρος του κώδικα παίρνει την εξής μορφή:

```
void phods_motion_estimation(const int current[N][M], const int previous[N][M],
    int vectors_x[N/B][M/B], int vectors_y[N/B][M/B])
{
    int x, y, i, j, k, l, p1, p2, q2, distx, disty, S, min1, min2, bestx, besty;

    distx = 0;
    disty = 0;

    //Data Reuse - Define frequently used data outside loops
    int mult_Bx, mult_By, Bx_vector, By_vector, Bx_vector_i, By_vector_i, mult_Bx_k;
    int upper_x = N/B;
    int upper_y = M/B;
    int min_init = 255*B*B;

    /*For all blocks in the current frame*/
    //Loop Unrolling Method
    for(x=0; x<upper_x; x++)
    {
        mult_Bx = B*x;
        for(y=0; y<upper_y; y++)
        {
            mult_By = B*y;
            OUTER_LOOP(4);
            OUTER_LOOP(2);
            OUTER_LOOP(1);
        }
    }
}
```

Μετά την εφαρμογή και του loop unrolling λαμβάνουμε τους παρακάτω χρόνους:

Table 5: Χρόνοι εκτέλεσης με loop merge, data reuse, loop unrolling

Μετρική	Χρόνος (s)
Μέσος χρόνος εκτέλεσης	0.000729263341
Μέγιστος χρόνος εκτέλεσης	0.000729270861
Ελάχιστος χρόνος εκτέλεσης	0.00072926248

Έχουμε λοιπόν και πάλι βελτίωση, η οποία συνολικά είναι της τάξης του **60.56%**, γεγονός που αποδεικνύει πως ο συνδυασμός όλων των παρπάνω μεθόδων επιταχύνει σημαντικά την εκτέλεση του προγράμματος και μειώνει τους κύκλους που απαιτούνται για την ολοκλήρωσή του.

Αξίζει να σημειώσουμε ότι δοκιμάσαμε και κάποιους άλλους μετασχηματισμούς, όπως το loop reversal στα loops που διατρέχουν τα x, y και τα k, l. Με τις αλλαγές αυτές όμως, δεν παρατηρήσαμε κάποια βελτίωση, γεγονός που οφείλεται στο ότι τα δεδομένα του αλγορίθμου (δηλαδή οι εικόνες) βρίσκονται κυρίως στη μνήμη Flash, η οποία δεν επωφελείται από τη βελτίωση της τοπικότητας αναφοράς. Η αλλαγή της σειράς εκτέλεσης των βρόχων δεν μειώνει τον αριθμό των προσπελάσεων ούτε το latency πρόσβασης, με αποτέλεσμα ο συνολικός χρόνος εκτέλεσης να παραμένει πρακτικά αμετάβλητος.

1.4

Στο ερώτημα αυτό πραγματοποιούμε διαδικασία **Design Space Exploration** στον κώδικα του ερωτήματος 3 με σκοπό την εύρεση του βέλτιστου μεγέθους μπλοκ B για την αρχιτεκτονική του συστήματος μας. Η διαδικασία βασίζεται σε εξαντλητική αναζήτηση για όλες τις τιμές του B που αποτελούν κοινούς διαιρέτες των διαστάσεων M και N των εικόνων. Για κάθε υποψήφια τιμή του B, η συνάρτηση εκτελέστηκε δέκα φορές και ως μετρική απόδοσης χρησιμοποιήσαμε τον μέσο όρο του χρόνου εκτέλεσης. Για την υλοποίηση της παραπάνω διαδικασίας τροποποιήσαμε όπως φαίνεται παρακάτω το σημείο του κώδικα που εκτελεί τις 10 επαναλήψεις, τροποποιώντας όπου χρειάζεται και την συνάρτηση που υλοποιεί τον αλγόριθμο ώστε να είναι συμβατή με τις αλλαγές μας. Χρησιμοποιήσαμε επίσης breakpoints για να παρατηρούμε κάθε φορά τον χρόνο εκτέλεσης για κάθε B.

```
void hal_entry(void) {
    // Code to initialize the DWT->CYCCNT register
    CoreDebug->DEMCR |= 0x01000000;
    ITM->LAR = 0xC5ACCE55;
    DWT->CYCCNT = 0;
    DWT->CTRL |= 1;
    /* Initialize your variables here */

    float mean_time, max_time, min_time;
    uint32_t start, end, cycles, max_cyc, min_cyc, tot_cycles;

    min_cyc = MAX_INT;
    max_cyc = 0;
    tot_cycles = 0;

    int B_matrix[] = {1, 2, 5, 10}; //common divisors of N and M;
    for (uint8_t ii = 0; ii < 4; ++ii){

        min_cyc = MAX_INT;
        max_cyc = 0;
        tot_cycles = 0;

        B = B_matrix[ii];
        int motion_vectors_x[N/B][M/B], motion_vectors_y[N/B][M/B];

        for(uint8_t it=0; it<10; ++it){
            /* Add timer code here */
            start = DWT->CYCCNT;
            phods_motion_estimation(current,previous,
            motion_vectors_x,motion_vectors_y, B_matrix[ii]);
            end = DWT->CYCCNT;

            cycles = end - start;

            if (max_cyc <= cycles)
            {
                max_cyc = cycles;
            }

            if(min_cyc >= cycles)
```

```

    {
        min_cyc = cycles;
    }

    tot_cycles += cycles;
}

max_time = (float) max_cyc / Fclk;
min_time = (float) min_cyc / Fclk;
mean_time = (float) tot_cycles / Fclk / 10.0;
tot_cycles = 0;
}

```

Στον παρακάτω πίνακα παρουσιάζουμε τον μέσο χρόνο εκτέλεσης για κάθε B . Από τα αποτελέσματα του πίνακα παρατηρείται ότι ο χρόνος εκτέλεσης της συνάρτησης `phods_motion_estimation` μειώνεται καθώς αυξάνεται το μέγεθος του μπλοκ B . Το σύστημα μας διαθέτει περιορισμένες cache (256 και 16 bytes) και μικρό prefetch buffer, γεγονός που καθιστά κρίσιμη τη βελτιστοποίηση της τοπικότητας δεδομένων. Για μεγαλύτερες τιμές του B , τα δεδομένα αποθηκεύονται και επαναχρησιμοποιούνται πιο αποτελεσματικά στη μνήμη, μειώνοντας το πλήθος των αναγνώσεων από τη Flash. Η καλύτερη επίδοση επιτυγχάνεται για $B=10$, με μέσο χρόνο εκτέλεσης ίσο με 0.00698s. Συγκριτικά με την τιμή $B=1$, η μείωση του χρόνου φτάνει περίπου το **50.9%**, γεγονός που επιβεβαιώνει τα όσα αναφέραμε παραπάνω για τις προσπελάσεις στη μνήμη. Αξίζει να σημειώσουμε ότι σε σχέση με την αρχική μορφή του αλγορίθμου για $B=10$ πετυχαίνουμε βελτίωση της τάξης του **62.27%**.

Table 6: Μέσος χρόνος εκτέλεσης της για διαφορετικές τιμές του B

Μέγεθος μπλοκ B	Μέσος χρόνος εκτέλεσης (s)
1	0.00142151667
2	0.000955766626
5	0.000752633344
10	0.000697712507

1.5

Στο ερώτημα αυτό πραγματοποιούμε και πάλι διαδικασία **Design Space Exploration** με σκοπό την εύρεση του βέλτιστου διδιάστατου μεγέθους μπλοκ (B_x, B_y) για την αρχιτεκτονική του επεξεργαστή μας. Για κάθε ζεύγος τιμών B_x και B_y , όπου το B_x αποτελεί διαμέτρο του N και το B_y διαμέτρο του M , ο αλγόριθμος `phods_motion_estimation` εκτελέστηκε 10 φορές και υπολογίστηκε ο μέσος όρος του χρόνου εκτέλεσης ως μετρική απόδοσης. Οι αλλαγές που κάναμε στο κύριο κομμάτι του κώδικα φαίνονται παρακάτω. Σημειώνουμε επίσης ότι έγιναν οι κατάλληλες αλλαγές στη συνάρτηση του αλγορίθμου ώστε να είναι συμβατή με τις αλλαγές.

```

void hal_entry(void) {
    // Code to initialize the DWT->CYCCNT register
    CoreDebug->DEMCR |= 0x01000000;
    ITM->LAR = 0xC5ACCE55;
    DWT->CYCCNT = 0;
    DWT->CTRL |= 1;

    /* Initialize your variables here */
    float mean_time, max_time, min_time;
    uint32_t start, end, cycles, max_cyc, min_cyc, tot_cycles;

    min_cyc = MAX_INT;

```

```

max_cyc = 0;
tot_cycles = 0;

int Bx_matrix[] = {1, 2, 5, 10}; //common divisors of N and M;
int By_matrix[] = {1, 2, 5, 10}; //common divisors of N and M;
float time_matrix[4][4];
for (uint8_t ii = 0; ii < 4; ++ii){
    for (uint8_t jj = 0; jj < 4; ++jj){

        tot_cycles = 0;

        Bx = Bx_matrix[ii];
        By = By_matrix[jj];
        int motion_vectors_x[N/Bx][M/By], motion_vectors_y[N/Bx][M/By];

        for(uint8_t it=0; it<10; ++it){
            /* Add timer code here */
            start = DWT->CYCCNT;
            phods_motion_estimation(current,previous,motion_vectors_x,
            motion_vectors_y, Bx, By);
            end = DWT->CYCCNT;

            cycles = end - start;

            tot_cycles += cycles;
        }
        mean_time = (float) tot_cycles / Fclk;
        time_matrix[ii][jj] = mean_time;
    }
}

```

Στον παρακάτω πίνακα φαίνονται συγκεντρωτικά τα αποτελέσματα για όλα τα ζεύγη B_x B_y :

Table 7: Μέσος χρόνος εκτέλεσης για ζεύγη διαστάσεων μπλοκ (B_x, B_y)

B_y	B_x			
	1	2	5	10
1	0.0141853793	0.0103588793	0.000806579553	0.00730129564
2	0.0126056671	0.000956583768	0.000774412928	0.00713733770
5	0.0116556669	0.000908896234	0.000755112898	0.00703883730
10	0.0113390004	0.000893000420	0.000748679601	0.000700604590

Από τα αποτελέσματα του πίνακα παρατηρείται ότι ο χρόνος εκτέλεσης μειώνεται σταδιακά όσο αυξάνονται οι διαστάσεις των μπλοκ B_x και B_y . Ο βέλτιστος συνδυασμός επιτυγχάνεται για $B_x=10$ και $B_y=10$, όπου ο μέσος χρόνος εκτέλεσης ανέρχεται σε περίπου 0.00070s. Σε σχέση με τον αρχικό αλγόριθμο, αυτό αντιστοιχεί σε συνολική βελτίωση της τάξης του **62.1%**. Το αποτέλεσμα αυτό επιβεβαιώνει ότι η αύξηση του μεγέθους των μπλοκ βελτιώνει την τοπικότητα δεδομένων, μειώνει τις προσπελάσεις στη μνήμη και αξιοποιεί πιο αποτελεσματικά τους διαθέσιμους πόρους του συστήματος. Για τον περιορισμό του χώρου αναζήτησης των διαστάσεων B_x και B_y , μπορεί να χρησιμοποιηθεί μια ευριστική προσέγγιση που εστιάζει αρχικά σε τετράγωνα μπλοκ ($B_x=B_y$). Στη συνέχεια, η αναζήτηση μπορεί να περιοριστεί γύρω από την περιοχή που εμφανίζει τον μικρότερο χρόνο εκτέλεσης, με μικρά βήματα αύξησης ή μείωσης των διαστάσεων. Με αυτόν τον τρόπο μειώνεται ο αριθμός των δοκιμών, χωρίς σημαντική απώλεια στην ακρίβεια της εύρεσης του βέλτιστου συνδυασμού.

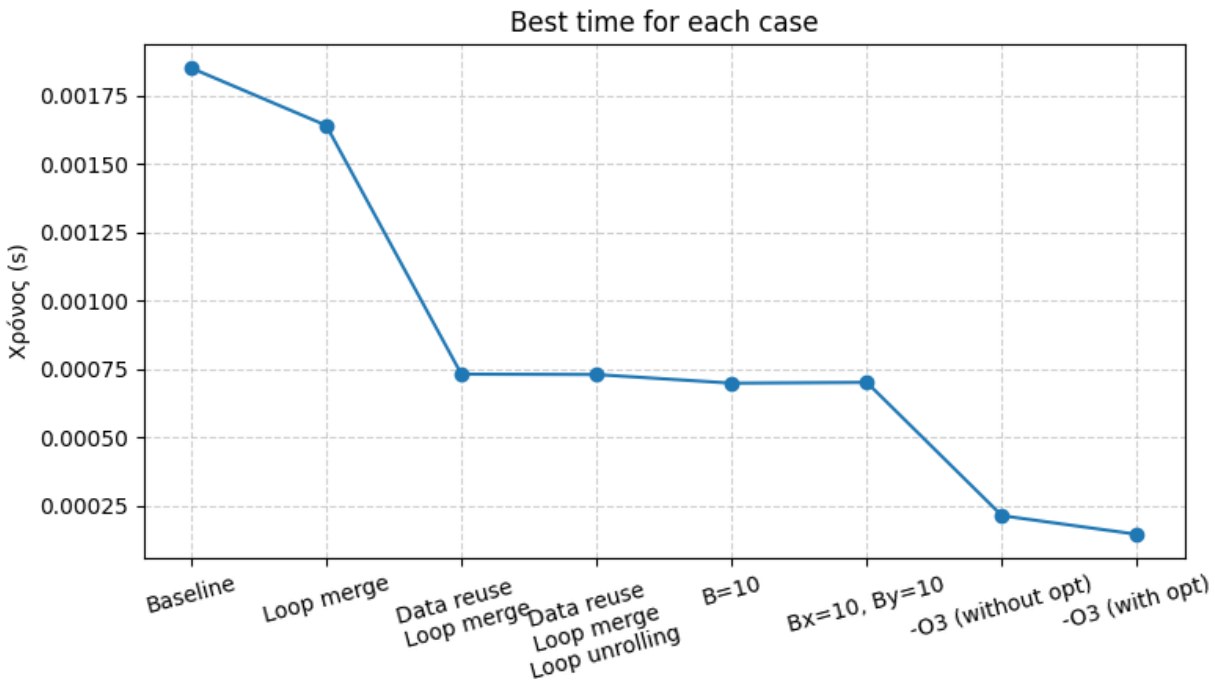
Κατά την εκτέλεση όλων των παραπάνω δοκιμών ορίσαμε αρχικά το flag του μεταγλωττιστή -O0, ώστε να απενεργοποιηθούν πλήρως τα αυτόματα optimizations και να απομονωθεί η επίδραση των δικών μας μετασχηματισμών. Ως επιπλέον δοκιμή, επαναμεταγλωττίσαμε με -O3 — ενεργοποιώντας επιθετικές βελτιστοποιήσεις και μετρήσαμε εκ νέου τους χρόνους εκτέλεσης του κώδικα χωρίς και με τις δικές μας βελτιστοποιήσεις. Τα αποτελέσματα που ακολουθούν παρουσιάζουν τη σύγκριση ανάμεσα στα δύο προφίλ μεταγλώττισης.

Table 8: Σύγκριση χρόνων εκτέλεσης με flag -O3

Περίπτωση	Μέσος χρόνος εκτέλεσης (s)	Κύκλοι (tot_cycles)
Χωρίς βελτιστοποιήσεις	0.000213409169	512182
Με βελτιστοποιήσεις	0.000145167497	348402

Η συνολική βελτίωση σε σχέση με τον αρχικό κώδικα φτάνει περίπου το **88.5%**, γεγονός που αποδεικνύει τη σημαντική συνεισφορά του compiler στις επιδόσεις μέσω του flag -O3. Ο μεταγλωττιστής εφαρμόζει εκτεταμένες βελτιστοποιήσεις, όπως inlining, loop unrolling και βελτιστοποίηση πρόσβασης στη μνήμη, μειώνοντας δραστικά τον χρόνο εκτέλεσης σε σχέση με την αρχική, μη βελτιστοποιημένη έκδοση του κώδικα

Τέλος, στο παρακάτω διάγραμμα line plot παρουσιάζονται συγκεντρωτικά όλοι οι χρόνοι εκτέλεσης των διαφορετικών δοκιμών, κρατώντας για κάθε κατηγορία τον βέλτιστο χρόνο στις περιπτώσεις όπου υπήρξαν μετρήσεις για διαφορετικές τιμές του B.



Από το διάγραμμα παρατηρείται σταδιακή μείωση του χρόνου εκτέλεσης καθώς εφαρμόζονται διαδοχικά οι βελτιστοποιήσεις, με τη μεγαλύτερη βελτίωση να επιτυγχάνεται στις εκδόσεις που συνδυάζουν τεχνικές επαναχρησιμοποίησης δεδομένων και βελτιστοποίησης βρόχων. Η χρήση του flag -O3 οδηγεί σε ακόμη χαμηλότερους χρόνους, αποδεικνύοντας την αποτελεσματικότητα των αυτόματων βελτιστοποιήσεων του μεταγλωττιστή.

Ζητούμενο 2ο

Στο 2ο ζητούμενο της άσκησης θα βελτιστοποιήσουμε την εφαρμογή `tables.c`, της οποίας η λειτουργία αφορά απλές προσπελάσεις και πράξεις με πίνακες. Με την βοήθεια του προσωμοιωτή `Gem5` θα εκτελέσουμε το πρόγραμμα σε δύο διαφορετικές αρχιτεκτονικές x86. Στην 1η αρχιτεκτονική έχουμε τον επεξεργαστή και την κύρια μνήμη, ενώ στην 2η επιπλέον L1 Instruction και Data caches και L2 cache.

2.1

Σημειώνουμε για αυτό το ζητούμενο ότι πρέπει να τροποποιήσουμε τον κώδικα του αρχείου `simple.py` ώστε να μπορεί να δέχεται σαν όρισμα ένα εκτελέσιμο, όπως είναι η εντολή που δίνεται στην εκφώνηση της άσκησης. Βοήθεια στο να το υλοποιήσουμε αποτελεί ο κώδικας του `two.level.py`, στο οποίο κάτι τέτοιο είναι δυνατό.

Στην αρχιτεκτονική χωρίς κρυφές μνήμες η εκτέλεση της εφαρμογής απαιτεί **858490254** κύκλους.

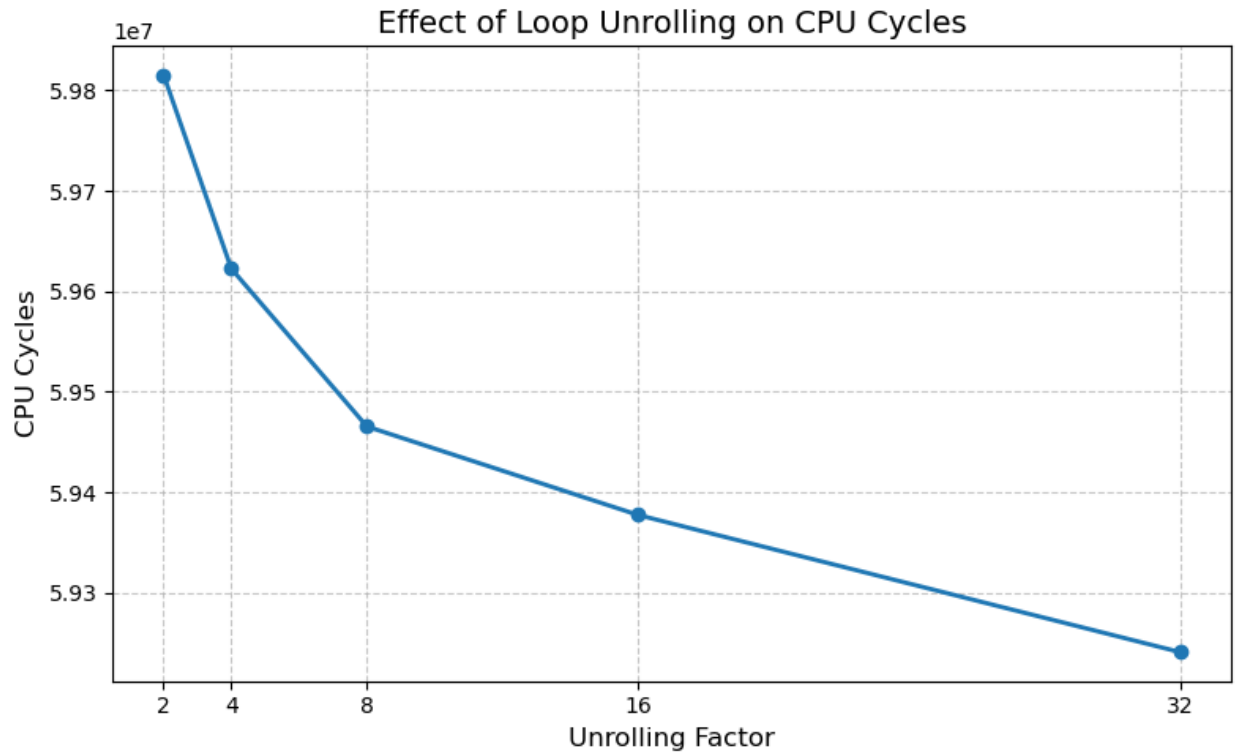
2.2

Στην 2η αρχιτεκτονική έχουμε L1 Data cache και L1 Instruction cache μεγέθους 8KB και L2 cache μεγέθους 128KB. Η εκτέλεση της εφαρμογής σε αυτήν την αρχιτεκτονική χρειάζεται **59829295** κύκλους.

Παρατηρούμε ότι για την 2η αρχιτεκτονική έχουμε λιγότερους κύκλους. Το αποτέλεσμα είναι λογικό, αφού στην 1η αρχιτεκτονική όταν ο επεξεργαστής χρειάζεται πρόσβαση σε εντολές ή δεδομένα, πρέπει να τα προσπελάσει από την κύρια μνήμη. Στην 2η αρχιτεκτονική, τα δεδομένα που έρχονται για υπολογισμό και εντολές που έχουν εκτελεστεί βρίσκονται στις αντίστοιχες κρυφές μνήμες (L1 ή/και L2). Γνωρίζουμε ότι η πρόσβαση στις κρυφές μνήμες είναι σημαντικά ταχύτερη από αυτήν στην κύρια μνήμη και για αυτό παρατηρούμε καλύτερα χρονικά αποτελέσματα στην 2η αρχιτεκτονική.

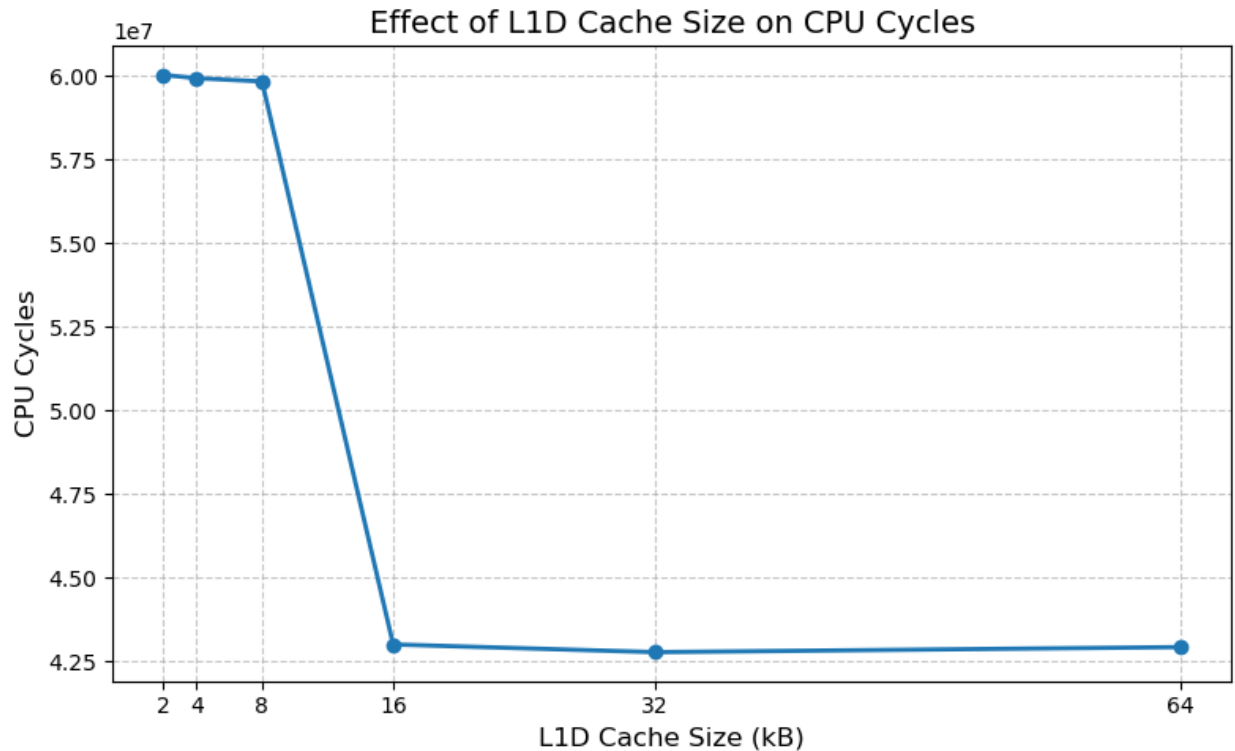
2.3

Για την 2η αρχιτεκτονική δοκιμάζουμε την επίδοση του προγράμματος για διαφορετικές τιμές unrolling factor. Παρουσιάζουμε το παρακάτω line plot:



Το unrolling factor είναι ο παράγοντας που καθορίζει όταν κάνουμε loop unrolling πόσες φορές το σώμα του αρχικού βρόγχου θα περιέχεται στον τελικό βρόγχο. Παρατηρούμε ότι για όλες τις πιθανές τιμές βελτιώνεται ο αριθμός των κύκλων του προγράμματος, αφού με το loop unrolling έχουμε λιγότερες εντολές branch και καλύτερο instruction level parallelism. Το πρόβλημα που δεν προκύπτει στην περίπτωση μας, αλλά θα μπορούσε να προκύψει από μεγάλο unrolling factor είναι ο κώδικας να μην "χωράει" στην cache (L1 instruction cache), με αποτέλεσμα τελικά να είχαμε χειρότερηση της απόδοσης.

Για την ίδια αρχιτεκτονική πειραματιζόμαστε με το μέγεθος της L1 Data cache, δημιουργούμε και παρουσιάζουμε τα αποτελέσματα στο παρακάτω line plot:



Μέχρι μέγεθος L1 data cache 16KB μπορούμε να παρατηρήσουμε ότι ο αριθμός των κύκλων μειώνεται, με μεγαλύτερη διαφορά από 8KB σε 16KB. Για μεγαλύτερο μέγεθος της L1 data cache παρατηρούμε ότι για 32KB υπάρχει μια μικρή βελτίωση και για 64KB μικρή χειροτέρευση σχετικά. Η βελτίωση μέχρι και τα 16KB συμβαίνει γιατί δεν μπορούν να χωρέσουν όλα τα δεδομένα στην L1 data cache και έτσι όσο αυξάνεται το μέγεθος της cache, περισσότερα δεδομένα μπορούν να βρισκονται εκεί με αποτέλεσμα να έχουμε ταχύτερη εκτέλεση. Για τα αποτελέσματα που βλέπουμε για μεγαλύτερο μέγεθος της cache, για τα δεδομένα του προγράμματός μας δεν χρειάζεται περισσότερη L1 cache από 32KB και για αυτό για 64KB βλέπουμε χειροτέρευση, γιατί ο στόχος ενός μεγαλύτερου μεγέθους cache είναι να μειωθούν τα cache misses. Ωστόσο, όταν, όπως στην περίπτωση μας δεν μειώνονται τα cache misses με μεγαλύτερο μέγεθος, έχουμε απλά αύξηση των κύκλων σε περίπτωση cache miss, καθώς χρειάζονται περισσότεροι κύκλοι για να ψάξουμε τα δεδομένα σε μια μεγαλύτερη cache. Οι επιπλέον αυτοί κύκλοι αποτελούν πρόβλημα μόνο σε περιπτώσεις που η μεγαλύτερη cache δεν φέρνει κάποια βελτίωση και το κόστος είναι μικρό, καθώς σε περίπτωση που η μεγαλύτερη cache βελτιώνει τα cache misses, το συνολικό κέρδος είναι σημαντικό.

Τα scripts που χρησιμοποιούμε για την εκτέλεση των παραπάνω μετρήσεων είναι αυτά που φαίνονται παρακάτω:

```
#!/bin/bash

output_file= unrolling_results.txt
> $output_file

for uf in 2 4 8 16 32; do
    echo Running simulation with unrolling factor = $uf
    build/X86/gem5.opt configs/learning_gem5/part1/two_level.py /gem5/
    tables_UF/tables_uf${uf}.exe --l1i_size=8kB --l1d_size=8kB --
    l2_size=128kB

    result_line=$(grep system.cpu.numCycles m5out/stats.txt)
```

```

{
    echo    unrolling factor = $uf
    echo    $result_line
    echo
} >> $output_file
done

output_file= l1d_results.txt
> $output_file

# List of L1 data cache sizes to test
for size in 2kB 4kB 8kB 16kB 32kB 64kB; do
    echo    Running simulation with L1D size = $size

    # Run gem5 with current L1D size
    build/X86/gem5.opt configs/learning_gem5/part1/two_level.py /gem5/
        tables_UF/tables.exe --l1i_size=8kB --l1d_size=$size --l2_size=128
        kB

    # Extract the numCycles line
    result_line=$(grep    system.cpu.numCycles    m5out/stats.txt)

    # Write results to file
    {
        echo    L1D size = $size
        echo    $result_line
        echo
    } >> $output_file
done

```

2.4

Ο στόχος μας στο ερώτημα αυτό είναι να βελτιστοποιήσουμε από κοινού το πρόγραμμά μας και την αρχιτεκτονική στην οποία θα εκτελεστεί. Επιθυμούμε να βρούμε τρόπο να ελασικοποιήσουμε το πλήθος κύκλων που απαιτούνται για την εκτέλεση του προγράμματος και την συνολική μνήμη που χρησιμοποιούμε, δηλαδή το άθροισμα των caches σε KB. Εξερευνούμε τον παρακάτω χώρο:

$L1D \text{ cache size} \in \{2 \text{ kB}, 4 \text{ kB}, 8 \text{ kB}, 16 \text{ kB}, 32 \text{ kB}, 64 \text{ kB}\}$
 $L1I \text{ cache size} \in \{2 \text{ kB}, 4 \text{ kB}, 8 \text{ kB}, 16 \text{ kB}, 32 \text{ kB}, 64 \text{ kB}\}$
 $L2 \text{ cache size} \in \{128 \text{ kB}, 256 \text{ kB}, 512 \text{ kB}, 1024 \text{ kB}\}$
 $\text{Unrolling factor} \in \{2, 4, 8, 16, 32\}$

Δημιουργούμε το παρακάτω script για να πραγματοποιήσουμε εξαντλητική αναζήτηση και αποθηκεύουμε τα αποτελέσματα για κάθε συνδυασμό σε ένα csv αρχείο:

```
#!/bin/bash
```

```

# Output CSV file
output_file= all_results.csv
echo l1d cache size,l1i cache size,l2 cache size,unrolling factor,number
    of cycles > $output_file

# Parameter lists
l1_sizes=( 2kB 4kB 8kB 16kB 32kB 64kB )
l2_sizes=( 128kB 256kB 512kB 1024kB )
unroll_factors=(2 4 8 16 32)

# Loop over all combinations
for l1i in ${l1_sizes[@]} ; do
    for l1d in ${l1_sizes[@]} ; do
        for l2 in ${l2_sizes[@]} ; do
            for uf in ${unroll_factors[@]} ; do

                echo Running: UF=$uf, L1I=$l1i, L1D=$l1d, L2=$l2

                # Run gem5
                build/X86/gem5.opt configs/learning_gem5/part1/two_level.py /gem5/
                    tables_UF/tables_uf${uf}.exe \
                    --l1i_size=$l1i --l1d_size=$l1d --l2_size=$l2

                # Extract just the numeric value of numCycles
                num_cycles=$(grep system.cpu.numCycles m5out/stats.txt | awk '{
                    print $2}')

                # Append results to CSV
                echo $l1d,$l1i,$l2,$uf,$num_cycles >> $output_file

            done
        done
    done
done

```

Ο χρόνος εκτέλεσης της εξαντλητικής αναζήτησης είναι **95m και 22.150s**.

Εφόσον έχουμε εξάγει όλα τα αποτελέσματα σε ένα csv αρχείο, το επόμενο βήμα μας είναι να βρούμε το Pareto Frontier. Θα το πραγματοποιήσουμε χρησιμοποιώντας την βιβλιοθήκη `paretoset` της python. Με το παρακάτω script σε python θα δημιουργήσουμε ένα pandas dataframe το οποίο περιλαμβάνει 1 στήλη με την συνολική μνήμη και 1 με τους συνολικούς κύκλους, όπως μας ζητείται.

```

import pandas as pd
from paretoset import paretoset
import matplotlib.pyplot as plt

def to_kb(size_str):
    size_str = size_str.strip().lower().replace('b', '')
    return int(size_str.replace('k', ''))

# Read the CSV
df = pd.read_csv( results/all_results.csv )

# Convert sizes to KB and compute total
df[ L1I_KB ] = df[ l1i cache size ].apply(to_kb)

```

```

df[ L1D_KB ] = df[ l1d cache size ].apply(to_kb)
df[ L2_KB ] = df[ l2 cache size ].apply(to_kb)

# Compute total memory in KB
df[ totalMemoryKB ] = df[ L1I_KB ] + df[ L1D_KB ] + df[ L2_KB ]

# Rename cycles column
df[ latencyCC ] = df[ number of cycles ]

df.to_csv( results/all_results_gen.csv , index=False)

mask = paretoiset(df[[ totalMemoryKB , latencyCC ]], sense=[ min , min ])
pareto_df = df[mask]

plt.figure(figsize=(10,6))

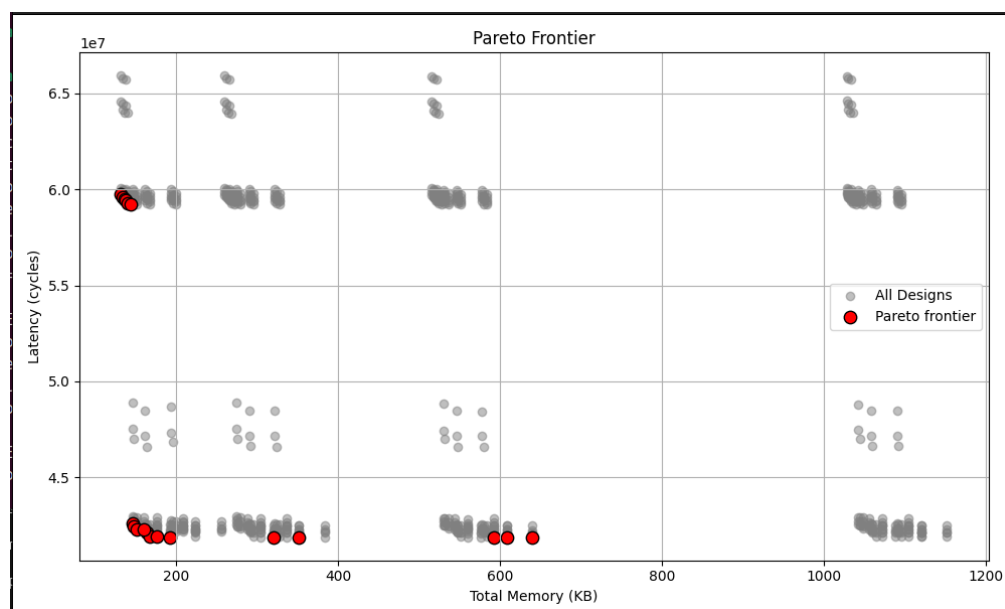
# Plot all points in light gray
plt.scatter(df[ totalMemoryKB ], df[ latencyCC ], color= gray , alpha=0.5, label= All )

# Plot Pareto frontier points in red
plt.scatter(pareto_df[ totalMemoryKB ], pareto_df[ latencyCC ], color='red', s=80, label= Pareto frontier )

plt.xlabel( Total Memory (KB) )
plt.ylabel( Latency (cycles) )
plt.title(f Pareto Frontier )
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

Έχοντας εντοπίσει το pareto frontier, με τον παραπάνω κώδικα το οπτικοποιούμε για να επαληθεύσουμε την ορθή λειτουργία. Έχουμε το παρακάτω αποτέλεσμα:



Μπορούμε να παρατηρήσουμε από την γραφική αναπαράσταση των αποτελεσμάτων μας ότι τα σημεία του Pareto Frontier κάνουν dominate τα υπόλοιπα σημεία και για κάθε σημείο του Pareto Frontier δεν υπάρχει σημείο που να το κάνει dominate. Σημειώνουμε ότι ως dominate εννοούμε ότι ένα σημείο A κάνει dominate ένα σημείο B, όταν είναι τουλάχιστον εξίσου καλό με το B σε μία κατηγορία και αυστηρά καλύτερο στην άλλη (καλύτερο = μικρότερη τιμή). Τα σημεία, λοιπόν, που αποτελούν το κόκκινο σύνολο θα είναι τα βέλτιστα trade-offs ανάμεσα σε χρόνο και συνολική μνήμη που προκύπτουν από την εξαντλητική αναζήτηση (υπάρχουν αναλυτικά σε πίνακα παρακάτω).

2.5

Στο επόμενο βήμα μας εκτελούμε τον κώδικα που μας δίνεται και υπολογίζει το Pareto Frontier με χρήση ενός γενετικού αλγορίθμου. Στους επόμενους πίνακες παρουσιάζουμε τα Pareto Frontiers για τις δύο μεθόδους. Σημειώνεται ότι η μέθοδος που χρησιμοποιείται τώρα χρειάζεται **3m και 38.537s**. Είναι σημαντικά ταχύτερη μέθοδος από την εξαντλητική αναζήτηση και συνολικά πραγματοποιεί 29 evaluations, αρκετά λιγότερα από την εξαντλητική αναζήτηση.

L1L_KB	L1D_KB	L2_KB	unrolling factor	totalMemoryKB	latencyCC
2	2	128	8	132	59724828
2	16	128	8	146	42591707
4	2	128	16	134	59579732
4	4	128	16	136	59484646
4	16	128	16	148	42472280
4	32	128	16	164	42146735
8	2	128	32	138	59427801
8	4	128	32	140	59281539
8	8	128	32	144	59241019
8	16	128	32	152	42315832
8	32	128	32	168	41928215
16	16	128	32	160	42300132
16	32	128	32	176	41912388
16	64	512	32	592	41889209
32	32	128	32	192	41898540
32	32	256	32	320	41896021
32	64	256	32	352	41892206
32	64	512	32	608	41889106
64	64	512	32	640	41888238

Table 9: Pareto Frontier για εξαντλητική αναζήτηση

L1L_KB	L1D_KB	L2_KB	unrolling factor	totalMemoryKB	latencyCC
4	16	128	16	148	42472280
16	32	128	8	176	42180765
4	4	128	8	136	59567374
32	32	256	32	320	41896021
32	32	128	32	192	41898540
32	32	512	32	576	41895278
4	8	128	16	140	59386648

Table 10: Pareto Frontier με γενετικό αλγόριθμο

Από την παραπάνω χρονική ανάλυση των δύο μεθόδων έχουμε παρατηρήσει ότι η μέθοδος με τον γενετικό αλγόριθμο είναι σημαντικά ταχύτερη από την μέθοδο που περιλαμβάνει εξαντλητική αναζήτηση. Ωστόσο, η εξαντλητική αναζήτηση βρίσκει ένα μεγαλύτερο Pareto Frontier. Η εξαντλητική αναζήτηση μπορεί να ελέγξει

αναλυτικά όλα τα σημεία και να εξάγει το πλήρες Pareto Frontier. Αντίθετα, ο γενετικός αλγόριθμος τρέχει ταχύτερα, αλλά το Pareto Frontier που βρίσκει είναι μικρότερο και δεν συμπίπτει απαραίτητα με κάποιο υποσύνολο αυτού που προκύπτει με την εξαντλητική αναζήτηση, αφού όπως βλέπουμε από τους πίνακες παραπάνω ένα παράδειγμα είναι το σημείο (176, 42180765) στο 2ο σύνολο που γίνεται dominate από το (164, 42146735) του 1ου (του συνόλου όλων των σημείων δηλαδή).

Συμπεραίνουμε για τις δύο μεθόδους, λοιπόν, ότι όταν έχουμε ένα μικρό σύνολο από δεδομένα μπορούμε με την εξαντλητική αναζήτηση εύκολα να προσδιορίσουμε το ακριβές Pareto Frontier. Όταν όμως το σύνολο δεδομένων είναι μεγαλύτερο ή έχουμε μεγαλύτερες διαστάσεις στα δεδομένα η εξαντλητική αναζήτηση θα απαιτεί σημαντικά μεγάλο χρόνο για να εκτελέσει όλους τους υπολογισμούς. Σε τέτοιες περιπτώσεις συμφέρει η επιλογή λύσεων με χρήση γενετικού αλγόριθμου, καθώς μπορούμε να πάρουμε αποτελέσματα σε πολύ λιγότερο χρόνο, με το tradeoff ότι το σύνολο που έχουμε δεν είναι εγγυημένο ότι είναι το σύνολο των πραγματικά βέλτιστων συνδυασμών παραμέτρων.